Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт Перспективной инженерии
Департамент цифровых и робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4
дисциплины «ООП»

Выполнил: Мендеш Пашкоал Педру
3 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и Вычислительная техника», направленность (профиль) «Программное обеспечение средств вычислительной техники и автоматизированных систем», очная форма обучения

_____
(подпись)

Руководитель практики:
Воронкин Роман А. доцент факультета цифровых, робототехнических систем и электроники института перспективной инженерии.

_____
(подпись)

Отчет защищен с оценкой _____ Дата защиты_____

Ставрополь, 2026 г.

Тема: наследование и полиморфизм языка Python

Цель работы: Приобретение навыков по созданию иерархии классов при написании программ с помощью языка программирования Python версии 3.x.

Методика выполнения работы

1. Изучен теоретический материал по теме наследования, полиморф изма и абстрактных классов.
2. Создан общедоступный репозиторий на GitHub с лицензией MIT, Ссылка на репозиторий: https://github.com/Pascoalpm/python-inheritance-polymorphism.git
3. Проект инициализирован с использованием современного менедж ера пакетов uv.
4. Создан файл pyproject.toml для управления зависимостями и наст ройки инструментов (ruff, black, isort, mypy, pytest).
5. Разработана иерархия классов Car и Lorry (Задание 1).
6. Разработан абстрактный базовый класс Figure и производные клас сы Rectangle, Circle, Trapezium (Задание 2).
7. Написаны юнит-тесты с использованием pytest.
8. Код отформатирован с помощью black и isort, проверен линтером ruff и mypy.
9. Выполнена фиксация изменений и отправка на GitHub с использо ванием ветвления (main / develop).

Индивидуальное задание

Задание 1 Иерархия классов: Car → Lorry

Создать базовый класс Car (машина), характеризующийся торговой маркой (строка), числом цилиндров, мощностью. Определить методы переназначения и изменения мощности. Создать производный класс Lorry (грузовик), характеризующийся также грузоподъемностью кузова. Определить функции переназначения марки и изменения грузоподъемности.

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

class Car:
    """Базовый класс: машина."""

    def __init__(self, brand: str, cylinders: int, power: float) -> None:
        self._brand = brand
        self._cylinders = cylinders
        self._power = power

    @property
    def brand(self) -> str:
        return self._brand

    @brand.setter
    def brand(self, value: str) -> None:
        if not value:
            raise ValueError("Марка не может быть пустой")
        self._brand = value

    @property
    def power(self) -> float:
        return self._power

    @power.setter
    def power(self, value: float) -> None:
        if value < 0:
            raise ValueError("Мощность не может быть отрицательной")
        self._power = value

    def info(self) -> None:
        print(
            f"Car: {self._brand}, cylinders: {self._cylinders}, "
            f"power: {self._power} HP"
        )

class Lorry(Car):
    """Производный класс: грузовик."""

    def __init__(
        self, brand: str, cylinders: int, power: float, capacity: float
    ) -> None:
        super().__init__(brand, cylinders, power)
        self._capacity = capacity

    @property
    def capacity(self) -> float:
        return self._capacity

    @capacity.setter
    def capacity(self, value: float) -> None:
        if value < 0:
            raise ValueError("Грузоподъёмность не может быть отрицательной")
        self._capacity = value

    def info(self) -> None:
        print(
            f"Lorry: {self._brand}, cylinders: {self._cylinders}, "
            f"power: {self._power} HP, capacity: {self._capacity} tons"
        )

def main() -> None:
    """Демонстрация работы классов."""
    print("Задание 1: Car → Lorry\n")

    car = Car("Toyota", 4, 150)
    car.info()

    car.power = 180
    print(f"После изменения мощности: {car.power} HP")

    lorry = Lorry("Volvo", 6, 400, 20)
```

```
    lorry.info()

    lorry.brand = "Scania"
    lorry.capacity = 25
    print("\nПосле изменений:")
    lorry.info()

if __name__ == "__main__":
    main()
```

```
Задание 1: Car → Lorry

Car: Toyota, cylinders: 4, power: 150 HP
После изменения мощности: 180 HP
Lorry: Volvo, cylinders: 6, power: 400 HP, capacity: 20 tons

После изменений:
Lorry: Scania, cylinders: 6, power: 400 HP, capacity: 25 tons
```

Рисунок 1 – Результат выполнения задания 1

Задание 2

Абстрактный базовый класс: Figure Создать абстрактный базовый класс Figure с абстрактными методами вычисления площади и периметра. Создать производные классы: · Rectangle (прямоугольник) · Circle (круг) · Trapezium (трапеция) со своими функциями площади и периметра. Самостоятельно определить, какие поля необходимы, какие из них можно задать в базовом классе, а какие — в производных. Площадь трапеции: S = (a + b) × h / 2

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import math
from abc import ABC, abstractmethod

class Figure(ABC):
    """Абстрактный базовый класс: фигура."""

    def __init__(self, name: str = "Figure") -> None:
        self._name = name

    @abstractmethod
    def area(self) -> float:
        """Абстрактный метод: площадь."""
        ...

    @abstractmethod
    def perimeter(self) -> float:
        """Абстрактный метод: периметр."""
        ...

    def info(self) -> None:
        print(
            f"{self._name}: area = {self.area():.2f}, "
            f"perimeter = {self.perimeter():.2f}"
        )

class Rectangle(Figure):
    """Прямоугольник."""
```

```python
    def __init__(self, width: float, height: float) -> None:
        super().__init__("Rectangle")
        self._width = width
        self._height = height

    def area(self) -> float:
        return self._width * self._height

    def perimeter(self) -> float:
        return 2 * (self._width + self._height)

class Circle(Figure):
    """Круг."""

    def __init__(self, radius: float) -> None:
        super().__init__("Circle")
        self._radius = radius

    def area(self) -> float:
        return math.pi * self._radius**2

    def perimeter(self) -> float:
        return 2 * math.pi * self._radius

class Trapezium(Figure):
    """Трапеция."""

    def __init__(
        self, a: float, b: float, h: float, c: float, d: float
    ) -> None:
        super().__init__("Trapezium")
        self._a = a
        self._b = b
        self._h = h
        self._c = c
        self._d = d

    def area(self) -> float:
        return (self._a + self._b) * self._h / 2

    def perimeter(self) -> float:
        return self._a + self._b + self._c + self._d

def main() -> None:
    """Демонстрация полиморфизма."""
    print("Задание 2: Абстрактный класс Figure\n")

    figures = [
        Rectangle(10, 5),
        Circle(7),
        Trapezium(6, 4, 3, 5, 5),
    ]

    for fig in figures:
        fig.info()

    print("\nПроверка isinstance")
    for fig in figures:
        print(f"{fig._name} is Figure: {isinstance(fig, Figure)}")

if __name__ == "__main__":
    main()
```

```
Rectangle: area = 50.00, perimeter = 30.00
Circle: area = 153.94, perimeter = 43.98
Trapezium: area = 15.00, perimeter = 20.00

Проверка isinstance
Rectangle is Figure: True
Circle is Figure: True
Trapezium is Figure: True
```

Рисунок 2 – Результат выполнения задания 2

Проверка качества кода и тестирование

Проект настроен с использованием современных инструментов разработки:

- pytest — для написания юнит-тестов.
- mypy — для строгой статической проверки типов.
- ruff и black — для линтинга и автоформатирования.

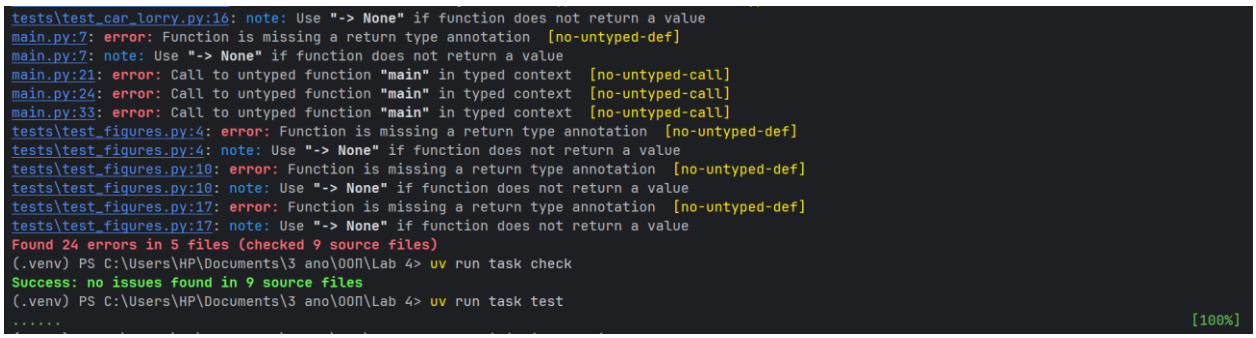


Рисунок 3 – Результаты прохождения тестов и проверки типов

Организация репозитория

Разработка проекта велась с использованием системы контроля версий Git. Весь исходный код, тесты и конфигурационные файлы (pyproject.toml, .gitignore) были загружены в публичный репозиторий на GitHub. Для организации рабочего процесса использовалась модель ветвления (ветки main и develop).

Ссылка на репозиторий: https://github.com/Pascoalpm/python-inheritance-polymorphism.git

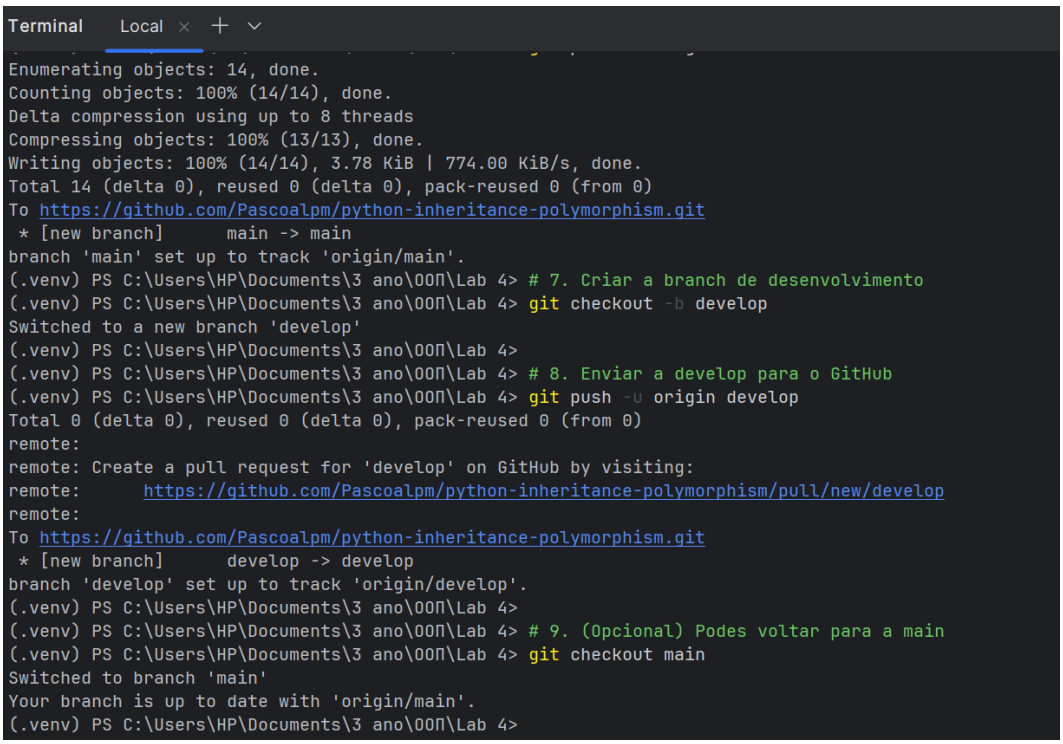


Рисунок 4 – Результат отправки изменений на сервер GitHub

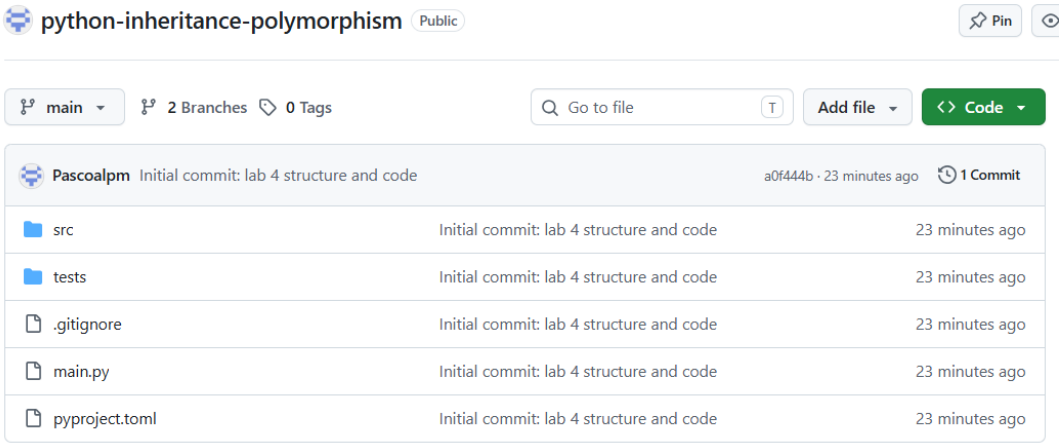


Рисунок 5 – Структура репозитория на GitHub

Вывод

В ходе выполнения лабораторной работы были успешно освоены механизмы наследования и полиморфизма в Python. Разработаны иерархии классов с использованием абстрактных базовых классов (Figure) и переопределением методов. Проект настроен с применением современных стандартов разработки (управление зависимостями через uv и pyproject.toml, статическая типизация mypy, автоформатирование black). Все задачи выполнены в полном объеме, код протестирован и опубликован на GitHub.

Ответы на контрольные вопросы

1. Что такое наследование и как оно реализовано в Python?

Наследование — это механизм ООП, позволяющий классу-потомку перенимать атрибуты и методы родительского класса. В Python оно реализуется путем указания имени родительского класса в скобках при объявлении дочернего: class Child(Parent):.

2. Что такое полиморфизм и как он реализован в Python?

Полиморфизм — это способность объектов с одинаковым интерфейсом (именами методов) иметь различную реализацию. В Python реализуется через переопределение методов (method overriding) в дочерних классах. Вызов метода зависит от того, объект какого конкретно класса его вызывает.

3. Что такое "утиная" типизация?

Это принцип динамической типизации в Python, при котором тип объекта менее важен, чем набор его методов и свойств. Если объект ведет себя как утка (крякает и плавает), то мы считаем его уткой. Проверяется не тип, а наличие нужных методов.

4. Каково назначение модуля abc?

Модуль abc (Abstract Base Classes) предоставляет инструменты для создания абстрактных базовых классов. Он позволяет определить интерфейс (набор методов), который обязательно должны реализовать дочерние классы.

5. Как сделать некоторый метод класса абстрактным?

Нужно унаследовать класс от ABC (из модуля abc) и использовать декоратор @abstractmethod перед методом.

6. Как сделать некоторое свойство класса абстрактным?

Использовать декоратор @property вместе с @abstractmethod:

```
@abstractmethod
@property
def my_prop(self): ...
```

7. Каково назначение функции isinstance?

Функция isinstance(obj, cls) проверяет, является ли объект obj экземпляром класса cls (или его наследником). Это полезно для безопасной проверки типов во время выполнения программы.